© Victor Tan 2025

Angular
Lab 2: Parent and Child Components Exercise Solution

1 LAB SETUP 1
2 GENERATING A NEW ANGULAR PROJECT 1
3 GENERATING AND USING A COMPONENT AS A CHILD 1
4 TRANSFERRING DATA FROM PARENT TO CHILD VIA @INPUT AND PROPERTY BINDING 1
5 TRANSFERRING DATA FROM CHILD TO PARENT VIA @OUTPUT AND EVENT BINDING 3
6 ACCESSING CHILD COMPONENT VIA TEMPLATE REFERENCE VARIABLE 5

1 Lab setup

2 Generating a new Angular project

3 Generating and using a component as a child

4 Transferring data from parent to child via @Input and property binding

Q1: Create a text field element and bind it using two way binding (NgModel) to a property in parent component

Q2: Create a @Input()property in the child component and display its contents via interpolation in the child template.

Q3: Bind the @Input property in the child with the parent property in the parent template within the selector tag of the child component

app.module.ts

```
 // Q1

/* Need to import FormsModule because we are using
NgModel in the template */
….
….
import { FormsModule } from '@angular/forms';
…..
…..



@Component({
  selector: 'app-root',

  // Need to include CommonModule and FormsModule
  // in imports metadata property
  imports: [RouterOutlet, CommonModule, FormsModule],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})

…
…
```

app.component.ts

```
  // Q1
  parentBoxContent = '';
```

app.component.html

```
<!-- Q1 -->
<label for="parentbox">Parent text box :</label><br>
<input type="text" id="parentbox" name="parentbox"
[(ngModel)]="parentBoxContent">
```

```
<!-- Q3 -->
<!-- Binding parent property to child property that has @Input decorator -->
<app-first-child [childCounter]="parentCounter"
[childTextBox]="parentBoxContent"></app-first-child>
```

first-child.component.ts

```
  //Q2
  @Input() childTextBox : string = '';
```

first-child.component.html

```
<!-- Q2 -->
<p>Child text box is: {{childTextBox}}</p>
```

5 Transferring data from child to parent via @Output and event binding

Q1. Create two radio buttons in the child template. Use event binding for these two buttons to a single child component method.

Q2. Declare a @Output property of type EventEmitter to transmit an event to the parent component. Use the EventEmitter in the component method from Q1 to emit the radio button value.

Q3. Create event binding in parent template to receive emitted event from child component.

Q4. Create a component method in parent component for the event binding in Q3. Store the received event in a new property in the parent component

Q5. Use interpolation to display the value of the new property in the parent template in a <p> element.

app.component.ts

```
  // Q4

 childRadioButton = '';

  processButtonChangeFromChild(val: string) {
    this.childRadioButton = val;
  }
```

app.component.html

```
<!-- Q5 -->
<p>Radio button value from child : {{childRadioButton}}</p>


<!-- Q3 -->
<!-- Binding parent property to child property that has @Input decorator -->
<!-- Event binding with target event being property from child with @Output
decorator -->
<app-first-child [childCounter]="parentCounter"
(textChanged)="processChangeFromChild($event)"
(buttonChanged)="processButtonChangeFromChild($event)"></app-first-child>
```

first-child.component.ts

```
  // Q2
  @Output()
 buttonChanged: EventEmitter<string> = new EventEmitter();


    // Q1, Q2
    processButtonChange(val: string) {
      console.log("The value of the clicked radio box is : " + val);
      this.buttonChanged.emit(val);
    }
```

first-child.component.html

```
<!-- Q1 -->
```

```
<input #firstradio type="radio" id="html" name="fav_language" value="HTML"
(click)="processButtonChange(firstradio.value)">
<label for="html">HTML</label><br>
<input #secondradio type="radio" id="css" name="fav_language" value="CSS"
(click)="processButtonChange(secondradio.value)">
<label for="css">CSS</label><br>
```

6 Accessing child component via template reference variable

Q1. Create two radio buttons in the child template. Use event binding for these two buttons to a single child component method and store the radio button value in a normal component property.

Q2. Add a template reference variable to the parent-child binding in the parent template and use this template reference variable to access the component property from Q1 and display it in the parent template via normal interpolation in a <p> element.

first-child.component.html

```
<!-- Q1 -->
<input #firstradio type="radio" id="html" name="fav_language" value="HTML"
(click)="processButtonChange(firstradio.value)">
<label for="html">HTML</label><br>
<input #secondradio type="radio" id="css" name="fav_language" value="CSS"
(click)="processButtonChange(secondradio.value)">
<label for="css">CSS</label><br>
```

first-child.component.ts

```
    // Q1
    radioButtonValue = '';

    processButtonChange(val: string) {
      console.log("The value of the clicked radio box is : " + val);
      this.radioButtonValue = val;
    }
```

app.component.html

```
<!-- Q2 -->
<p>Radio button value  : {{myChild.radioButtonValue}}</p>


<app-first-child #myChild [childCounter]="parentCounter"
(counterChanged)="parentCounter=$event"></app-first-child>
```